CAHIER DES CHARGES

Plateforme Centralisée AIOps

Automatisation, Supervision Prédictive, Cybersécurité et Gestion de Parc pour l'Administration Système & Réseau

Diplôme préparé : BTS SIO option SISR — également compatible CIEL / CPR
Cadre : Épreuve E6 — Projet Technique / Mémoire de soutenance
Technologies : Python (FastAPI) · React (Vite / Tailwind CSS) · SQLite / PostgreSQL · NetDevOps · Machine Learning

Version enrichie et maturée — Document de spécifications détaillées
Août 2026 (stack technique mise à jour en octobre 2026)

Sommaire

1. Contexte, problématique et maturation du projet 4

1.1 Rappel du contexte . . . 4
1.2 Travail de maturation réalisé . . . 4
1.3 Proposition de valeur reformulée . . . 4
1.4 Solutions existantes et positionnement . . . 4

2. Objectifs du projet 5

2.1 Objectif général . . . 5
2.2 Objectifs spécifiques (SMART) . . . 5
2.3 Objectifs pédagogiques (cadre BTS) . . . 6

3. Périmètre fonctionnel 6

3.1 Ce qui est inclus dans le projet (in scope) . . . 6
3.2 Ce qui est explicitement hors périmètre (out of scope) . . . 6

4. Architecture globale de la solution 7

4.1 Vue d'ensemble . . . 7
4.2 Schéma logique . . . 7
4.3 Schéma technique simplifié (flux de données) . . . 7

5. Spécifications fonctionnelles et techniques détaillées par module 8

5.1 Module Sécurité — Détection d'anomalies dans les journaux . . . 8
5.2 Module Supervision — Prédiction de panne des ressources . . . 8
5.3 Module NetDevOps — Audit et correction assistée des configurations réseau . . . 9
5.4 Module Gestion de parc informatique . . . 9
5.5 Module Reporting & Administration . . . 10
5.6 Module Assistant IA conversationnel (évolution proposée) . . . 11

6. Modèle de données (vue logique) 11

7. Exigences non fonctionnelles 12

8. Stack technique & charte graphique 12

8.1 Composants technologiques . . . 12
8.2 Charte graphique de l'interface (validée) . . . 13

9. Gestion des utilisateurs, rôles et sécurité applicative 14

9.1 Matrice des droits par rôle . . . 14
9.2 Exigences de sécurité applicative . . . 14

10. Plan de tests et recette 15

10.1 Niveaux de test . . . 15
10.2 Scénarios de test représentatifs . . . 15

11. Gestion de projet 15

11.1 Méthodologie retenue . . . 15

11.2 Planning prévisionnel (exemple sur 5 mois) 15
11.3 Budget prévisionnel (projet de laboratoire, sans matériel dédié) 16
11.4 Analyse des risques 16
12. Livrables attendus **16**
13. Perspectives d'évolution (hors périmètre de soutenance) **16**
14. Glossaire **17**

1. Contexte, problématique et maturation du projet

1.1 Rappel du contexte

Les infrastructures informatiques actuelles génèrent un volume considérable de données (journaux systèmes, métriques réseau, flux SNMP, fichiers de configuration) qu'il devient impossible de superviser manuellement au sein d'une équipe informatique restreinte, typique des PME/ETI ou des collectivités. L'administration système et réseau conventionnelle se heurte à trois limites structurelles : une posture réactive où l'intervention n'a lieu qu'après l'incident, une cécité face aux cybermenaces noyées dans le volume des journaux d'événements, et un risque élevé d'erreurs de configuration lors des opérations manuelles sur les équipements réseau.

1.2 Travail de maturation réalisé

Le dossier initial posait les fondations d'une plateforme à trois modules (Sécurité, Supervision, NetDevOps) puis une version enrichie a ajouté un module de gestion de parc et des fonctions transverses (reporting, utilisateurs, notifications). Ce travail de brainstorming approfondit encore la solution afin d'en faire un projet de soutenance BTS complet et cohérent, en couvrant à la fois la dimension produit (fonctionnalités), la dimension ingénierie logicielle (architecture, API, base de données, sécurité de la plateforme elle-même, CI/CD) et la dimension gestion de projet (méthodologie, planning, budget, risques, indicateurs de réussite).

Axes de maturation ajoutés

- **Module 6 — Assistant IA conversationnel :** un chatbot adossé à un LLM local (Ollama) permettant d'interroger la plateforme en langage naturel (« Quels serveurs risquent une saturation disque cette semaine ? »).
- **Sécurisation de la plateforme elle-même :** authentification forte (MFA), chiffrement des secrets, gestion fine des rôles (RBAC), journal d'audit infalsifiable — la plateforme qui protège l'infrastructure doit être elle-même exemplaire en matière de sécurité.
- **Architecture découplée API REST :** séparation stricte backend API REST (FastAPI) / frontend SPA (React + Vite + Tailwind CSS) pour garantir une réactivité optimale, une reproduction fidèle des maquettes et une évolutivité vers de futurs clients.
- **Conteneurisation Docker** et scripts docker-compose pour une installation reproductible du lab de démonstration et faciliter la portabilité pédagogique.
- **Base de données relationnelle structurée** (SQLite en environnement local / démonstration pour une portabilité totale sans dépendance de serveur SGBD tiers, avec compatibilité PostgreSQL via l'ORM SQLAlchemy).
- **Démarche de gestion de projet :** méthodologie Agile/Scrum adaptée au contexte BTS, backlog, planning en jalons, budget prévisionnel, matrice des risques.
- **Conformité et éthique :** prise en compte du RGPD pour les données de journalisation (anonymisation des adresses IP internes, durée de conservation), et charte d'usage de l'IA.
- **Observabilité et exploitation :** sauvegardes automatisées de la base, supervision de la plateforme elle-même, procédure de restauration après sinistre (PRA simplifié).

1.3 Proposition de valeur reformulée

> *« La plateforme AIOps transforme l'administration système et réseau d'une gestion réactive, cloisonnée et manuelle vers un pilotage unifié, prédictif, sécurisé et partiellement automatisé, tout en restant un projet réalisable et démontrable dans le cadre d'un BTS. »*

1.4 Solutions existantes et positionnement

Le projet ne prétend pas inventer une catégorie d'outils inexistante : le marché de l'AIOps et de la supervision open-source est mature. Cette section situe la plateforme Sentinelle par rapport à l'existant, afin de justifier sa valeur ajoutée dans un contexte de projet de soutenance BTS.

Solutions commerciales de référence (marché AIOps)

Les grandes plateformes commerciales telles que Datadog, Dynatrace, Splunk ITSI, IBM Cloud Pak for AIOps ou ScienceLogic SL1 dominent le marché de l'observabilité pilotée par IA. Ces solutions sont passées en 2026 d'une logique prédictive à une logique agentique : elles corrèlent des milliers d'alertes en un nombre restreint d'incidents, identifient la cause racine, et vont parfois jusqu'à exécuter automatiquement des correctifs sans intervention humaine. Leur coût de licence et leur fonctionnement en boîte noire les rendent toutefois inadaptés à un contexte pédagogique et budgétaire de BTS.

Outils open-source comparables, module par module

Module Sentinelle	Équivalents open-source	Limite par rapport à Sentinelle
Supervision (§5.2)	Zabbix, Netdata, Prometheus/Grafana, LibreNMS	Bon sur la métrique brute, mais sans prédiction de panne (Time-To-Failure) nativement intégrée
Sécurité (§5.1)	Wazuh, suite ELK/Elastic Security, Zeek	Détection performante mais configuration lourde, peu orientée détection non supervisée simple
NetDevOps (§5.3)	Ansible (remédiation), Nipper, Cisco DNA Center	Peu d'outils combinent extraction Netmiko et audit assisté par LLM local selon un référentiel CIS
Gestion de parc (§5.4)	LibreNMS, NetBox	Inventaire solide mais non consolidé avec les scores de sécurité et de supervision

Valeur ajoutée du projet Sentinelle

- **Unification :** chaque outil cité ci-dessus excelle sur un seul périmètre (supervision, sécurité ou réseau). Sentinelle les réunit dans une base de données et un dashboard uniques, répondant directement au cloisonnement identifié en §1.1.
- **Audit réseau assisté par LLM local :** la combinaison Netmiko + LLM local (Ollama) prompté selon un référentiel CIS Benchmark, avec validation humaine obligatoire avant application, est peu répandue dans l'écosystème open-source généraliste.
- **Coût et transparence pédagogique :** une stack 100 % open-source, documentée et expliquée de bout en bout, à l'opposé des plateformes commerciales en boîte noire.
- **Portée volontairement raisonnable :** le projet ne cherche pas à concurrencer Datadog ou Splunk, mais à démontrer, à l'échelle d'un projet BTS, la faisabilité d'une approche AIOps unifiée et maîtrisée de bout en bout par l'étudiant.

> *« Sentinelle ne cherche pas à remplacer Zabbix ou Wazuh, mais à démontrer comment unifier supervision, sécurité et NetDevOps dans une seule plateforme pilotée par l'IA — une approche que seules les solutions AIOps commerciales proposent aujourd'hui, à un coût très différent. »*

2. Objectifs du projet

2.1 Objectif général

Concevoir, développer et démontrer une plateforme web centralisée capable de superviser, sécuriser, automatiser et documenter l'administration d'une infrastructure système et réseau hétérogène, en s'appuyant sur des techniques d'intelligence artificielle et d'apprentissage automatique légères, exécutables sur un poste ou un serveur de laboratoire standard.

2.2 Objectifs spécifiques (SMART)

N°	Objectif spécifique	Indicateur de réussite
O1	Détecter les anomalies de connexion (brute-force, scan) dans les journaux systèmes	Taux de détection ≥ 90 % sur jeu de test simulé, faux positifs < 10 %
O2	Prédire la saturation des ressources serveur avant l'incident	Prédiction du Time-To-Failure avec un écart < 20 % sur scénario simulé

N°	Objectif spécifique	Indicateur de réussite
O3	Auditer automatiquement les configurations réseau selon un référentiel CIS	100 % des équipements du lab audités, rapport généré en < 30 s / équipement
O4	Centraliser l'inventaire du parc informatique	Découverte automatique de 100 % des équipements SNMP actifs du lab
O5	Offrir un tableau de bord unique et un système d'alertes multicanal	Alerte critique reçue en < 60 s après détection (mail/Telegram/Discord)
O6	Garantir la sécurité et la traçabilité de la plateforme elle-même	Authentification MFA active, journal d'audit couvrant 100 % des actions sensibles

2.3 Objectifs pédagogiques (cadre BTS)

- Démontrer la maîtrise des compétences réseau/système (SISR) : SSH, SNMP, VLAN, sécurisation d'équipements Cisco.
- Démontrer la maîtrise du développement logiciel : architecture Python, API REST, base de données, tests.
- Illustrer une utilisation raisonnée de l'IA/ML appliquée à l'exploitation informatique (AIOps).
- Mettre en œuvre une démarche de projet complète : cahier des charges, planning, gestion des risques, recette.

3. Périmètre fonctionnel

3.1 Ce qui est inclus dans le projet (in scope)

Domaine	Inclus dans la version de soutenance
Supervision	Collecte SSH/SNMP, prédiction de saturation par régression, alertes seuils
Sécurité	Parsing des journaux (Syslog/Auth.log), détection d'anomalies par Isolation Forest
NetDevOps	Extraction de configuration via Netmiko, audit CIS Benchmark assisté par LLM local
Gestion de parc	Découverte SNMP, fiche équipement, historique, score de santé
Reporting / Admin	Dashboard React (Vite + Tailwind CSS), rapports PDF, gestion des rôles, notifications
Assistant IA	Chatbot de consultation en langage naturel sur les données déjà collectées

3.2 Ce qui est explicitement hors périmètre (out of scope)

- Le déploiement en environnement de production réel (le projet reste démontré sur maquette GNS3/EVE-NG et VMs).
- La correction automatique sans validation humaine des configurations réseau (l'IA suggère, l'administrateur valide).
- La gestion multi-clients / multi-tenant de type SaaS (proposée uniquement en perspective d'évolution, §12).
- La certification officielle ISO 27001 ou l'audit de sécurité tiers (hors cadre temporel du BTS).
- Le support d'environnements Cloud publics (AWS/Azure/GCP) en version 1 — piste d'évolution possible.

4. Architecture globale de la solution

4.1 Vue d'ensemble

L'architecture retenue est modulaire et découplée : chaque module métier (Sécurité, Supervision, NetDevOps, Parc) constitue un service Python autonome qui alimente une base de données centrale. Un backend API REST (FastAPI) expose les données et les actions aux interfaces (dashboard web React, assistant IA conversationnel, futurs clients). Cette séparation facilite les tests unitaires, l'évolutivité et la maintenance — chaque module peut être développé, testé et présenté indépendamment durant la soutenance.

4.2 Schéma logique

Couche	Composants	Rôle
Collecte	Agents SSH/SNMP, lecture Syslog/Auth.log, Netmiko	Récupérer les données brutes sur les serveurs et équipements réseau
Traitement / IA	Isolation Forest, Régression / séries temporelles, LLM local (Ollama)	Analyser, détecter, prédire, auditer
Persistance	SQLite par défaut (zéro dépendance de service pour la soutenance), compatible PostgreSQL via SQLAlchemy, sauvegardes de configuration	Conserver historique, alertes, prédictions, comptes utilisateurs
Services / API	Backend FastAPI (endpoints REST, authentification JWT)	Exposer les données et déclencher les actions
Présentation	Dashboard Single Page Application React (Vite + Tailwind CSS), chatbot IA intégré, rapports PDF, notifications	Restituer l'information aux utilisateurs
Transverse	Authentification, RBAC, journal d'audit, planification (scheduler)	Sécuriser et orchestrer l'ensemble

4.3 Schéma technique simplifié (flux de données)

Sources	Serveurs Linux / Windows · Switchs et routeurs Cisco (GNS3/EVE-NG) · Postes clients / équipements SNMP
Collecte	Syslog / Auth.log · SNMP / SSH / Netmiko · Découverte SNMP
Modules métier	Module Sécurité (Isolation Forest) · Module Supervision (Régression / TTF) · Module Gestion de parc (Inventaire / santé)
Module NetDevOps + LLM local	Audit CIS, correctifs proposés
Base de données centrale	SQLite par défaut, compatible PostgreSQL via SQLAlchemy — toutes les données consolidées
API REST — FastAPI	Authentification JWT, gestion des rôles RBAC
Interfaces	Dashboard React + Vite (Tailwind CSS / Recharts / Plotly) · Assistant IA (chat en lecture seule) · Notifications & rapports PDF / mail / Telegram

Les flux descendent d'une ligne à la suivante : les sources alimentent la collecte, puis les modules métier, la base centrale, l'API et enfin les interfaces.

5. Spécifications fonctionnelles et techniques détaillées par module

5.1 Module Sécurité — Détection d'anomalies dans les journaux

Objectif : identifier automatiquement les comportements suspects (force brute, scan de ports, connexions anormales) à partir des journaux systèmes, sans dépendre de règles statiques exhaustives.

Fonctionnement détaillé

- Collecte en flux ou par lot des fichiers Syslog et Auth.log (agents légers ou lecture distante SSH).
- Parsing par expressions régulières pour extraire : horodatage, adresse IP source, utilisateur ciblé, type d'évènement (échec, succès, déconnexion).
- Construction d'indicateurs par fenêtre glissante : fréquence de requêtes/minute, taux d'échecs d'authentification, nombre d'IP distinctes sollicitant un même service.
- Passage des indicateurs dans le modèle de détection d'anomalies ; scoring et classification (normal / suspect / critique).
- Génération d'une alerte horodatée et archivage dans l'historique pour analyse a posteriori.

Apport de l'intelligence artificielle : le modèle Isolation Forest (scikit-learn) isole les observations atypiques par partitionnement aléatoire récursif : moins de coupures nécessaires pour isoler un point signale une anomalie. Ce choix non supervisé évite d'avoir à disposer d'un jeu de données déjà labellisé en attaques, adapté à un contexte de laboratoire BTS où les incidents réels sont rares.

Données en entrée	Données / actions en sortie
Fichiers Syslog/Auth.log (flux ou export), liste blanche d'IP internes, seuils de sensibilité paramétrables	Alertes classées par sévérité (info/warning/critique), fiche d'incident horodatée, export CSV/PDF, notification temps réel

Critères d'acceptation

- ✓ Taux de détection ≥ 90 % sur un scénario de brute-force simulé (ex. hydra/medusa en environnement isolé).
- ✓ Faux positifs inférieurs à 10 % sur trafic normal simulé.
- ✓ Temps de traitement inférieur à 2 secondes pour un lot de 10 000 lignes de journal.

Technologies retenues : Python 3.11, pandas, numpy, scikit-learn (IsolationForest), regex, APScheduler pour l'analyse périodique.

5.2 Module Supervision — Prédiction de panne des ressources

Objectif : anticiper la saturation des ressources critiques (CPU, RAM, disque, bande passante) en estimant une date probable de rupture de service (« Time-To-Failure »).

Fonctionnement détaillé

- Interrogation périodique (toutes les N minutes, configurable) des serveurs et équipements réseau via SSH (psutil côté agent) et SNMP (OID standards : hrStorage, ifInOctets, etc.).
- Constitution d'une série temporelle par métrique et par équipement, stockée en base.
- Ajustement d'un modèle de régression (linéaire ou polynomiale légère / lissage exponentiel) sur la tendance récente de consommation.
- Extrapolation jusqu'au seuil critique défini (ex. 95 % d'occupation disque) pour estimer le Time-To-Failure, exprimé en jours/heures.
- Déclenchement d'une alerte préventive lorsque le TTF passe sous un seuil configurable (ex. 48 h).

Apport de l'intelligence artificielle : une régression linéaire (ou lissage de séries temporelles de type Holt) est appliquée sur les N dernières observations pour projeter la tendance. Le choix d'un modèle simple, interprétable et peu coûteux en ressources correspond aux contraintes d'un environnement de démonstration BTS, tout en restant explicable devant un jury.

Données en entrée	Données / actions en sortie
Métriques CPU/RAM/disque/réseau (psutil, SNMP), horodatage, seuils critiques par équipement	Courbe de tendance et de prédiction, estimation du Time-To-Failure, alerte préventive, historique des métriques exploitable pour capacity planning

Critères d'acceptation

- ✓ Écart entre TTF prédit et TTF réel inférieur à 20 % sur un scénario de remplissage disque simulé.
- ✓ Rafraîchissement du dashboard en moins de 5 secondes après collecte.
- ✓ Historique conservé sur une profondeur paramétrable (30 jours par défaut).

Technologies retenues : Python, psutil, pysnmp, pandas, scikit-learn (LinearRegression) ou statsmodels (Holt-Winters), APScheduler.

5.3 Module NetDevOps — Audit et correction assistée des configurations réseau

Objectif : réduire le risque de failles de sécurité liées à des configurations réseau erronées ou obsolètes, via un audit automatisé et des recommandations de correction.

Fonctionnement détaillé

- Connexion SSH automatisée aux équipements (Netmiko / Nornir) et extraction de la configuration active (show running-config).
- Normalisation du texte de configuration puis soumission à un modèle de langage local (via Ollama) prompté selon un référentiel de type CIS Benchmark (mots de passe en clair, SNMP en v1/v2 non sécurisé, services inutiles actifs, absence de bannière légale, etc.).
- Le LLM retourne une liste de constats classés par criticité et un projet de script de correction (commandes Cisco IOS), présenté à l'administrateur pour validation avant application.
- Sauvegarde automatique de la configuration avant toute modification, avec possibilité de restauration.

Apport de l'intelligence artificielle : le LLM local (exécuté via Ollama, donc sans envoi de données sensibles vers un service externe) joue un rôle d'assistant d'audit : il structure sa réponse selon un gabarit imposé (JSON) associant chaque anomalie détectée à une règle du référentiel CIS et à une commande de correction. L'humain reste décisionnaire : aucune commande n'est appliquée automatiquement sans validation.

Données en entrée	Données / actions en sortie
Configuration active de l'équipement (texte), référentiel CIS Benchmark (règles pré-enregistrées)	Rapport d'audit structuré, score de conformité par équipement, script de correction proposé, sauvegarde de configuration horodatée

Critères d'acceptation

- ✓ 100 % des équipements du lab audités sans erreur de connexion sur trois exécutions successives.
- ✓ Temps d'audit inférieur à 30 secondes par équipement.
- ✓ Chaque anomalie remontée est rattachée à une règle CIS identifiable et documentée.

Technologies retenues : Python, Netmiko, Nornir, Ollama (modèle type Llama 3 / Mistral en local), gabarit de prompt structuré, PyYAML pour le référentiel de règles.

5.4 Module Gestion de parc informatique

Objectif : constituer et maintenir à jour un inventaire centralisé des équipements système et réseau, avec un score de santé synthétique par équipement.

Fonctionnement détaillé

- Découverte automatique des équipements actifs par balayage SNMP sur les plages d'adresses définies.

- Récupération des informations descriptives : adresse IP, adresse MAC, système/OS, version de firmware, temps de fonctionnement (uptime).
- Calcul d'un score de santé agrégeant disponibilité, charge moyenne, nombre d'alertes récentes et conformité de configuration (issue du module NetDevOps).
- Mise à disposition d'une fiche équipement détaillée avec historique des alertes, audits et actions.

Apport de l'intelligence artificielle : l'IA intervient ici de façon indirecte, en consolidant les scores produits par les modules Sécurité, Supervision et NetDevOps pour calculer une note de santé pondérée par équipement, facilitant la priorisation des interventions.

Données en entrée	Données / actions en sortie
Plages d'adresses IP à scanner, réponses SNMP des équipements, résultats des autres modules	Inventaire à jour (fiches équipement), score de santé (0 à 100), historique consolidé, alertes d'équipements disparus du réseau

Critères d'acceptation

- ✓ Découverte de 100 % des équipements SNMP actifs présents dans le lab de démonstration.
- ✓ Mise à jour de l'inventaire en moins de 5 minutes après ajout d'un nouvel équipement.
- ✓ Score de santé recalculé automatiquement à chaque nouvelle donnée entrante.

Technologies retenues : Python, pysnmp, APScheduler, SQLite / PostgreSQL (via SQLAlchemy) pour la persistance de l'inventaire.

5.5 Module Reporting & Administration

Objectif : fournir une interface de pilotage unique, des rapports exploitables et une gestion rigoureuse des accès à la plateforme.

Fonctionnement détaillé

- Tableau de bord React (Vite + Tailwind CSS) présentant les indicateurs globaux (état du parc, alertes en cours, prédictions à venir, score de conformité moyen) avec visualisations interactives (Recharts / Plotly).
- Génération automatique de rapports PDF d'audit et de synthèse périodique (hebdomadaire/mensuelle), incluant recommandations priorisées.
- Gestion des comptes utilisateurs selon trois profils (Administrateur, Technicien, Visiteur) avec droits différenciés (lecture seule, actions correctives, administration complète).
- Journal d'activité (audit trail) enregistrant chaque connexion et chaque opération sensible (modification de configuration, changement de droits).
- Notifications multicanal configurables : e-mail (SMTP), Telegram, Discord, selon la criticité de l'évènement.

Apport de l'intelligence artificielle : cette couche exploite les sorties des modules IA sans en ajouter de nouvelle : son rôle est de restituer, prioriser et rendre actionnable l'information produite en amont.

Données en entrée	Données / actions en sortie
Données consolidées de tous les modules, actions des utilisateurs, préférences de notification	Dashboard interactif, rapports PDF, journal d'audit, notifications envoyées, comptes utilisateurs gérés

Critères d'acceptation

- ✓ Alerte critique notifiée sur au moins un canal en moins de 60 secondes.
- ✓ Rapport PDF généré automatiquement sans intervention manuelle, à fréquence paramétrable.
- ✓ Journal d'audit couvrant 100 % des actions de modification (non purgeable par un compte Technicien).

Technologies retenues : React 18/19, Vite, Tailwind CSS, Recharts / Plotly, ReportLab (génération PDF via FastAPI), smtplib, API Telegram Bot, Webhook Discord.

5.6 Module Assistant IA conversationnel (évolution proposée)

Objectif : permettre à un administrateur d'interroger la plateforme en langage naturel plutôt que de naviguer manuellement dans les tableaux de bord.

Fonctionnement détaillé

- Interface de chat interactive intégrée à la SPA React (composant drawer / modal dédié), connectée via FastAPI au LLM local.
- Le LLM ne répond pas « librement » : il génère une requête structurée (function calling / requête SQL paramétrée) exécutée par le backend sur la base de données, puis reformule le résultat en langage naturel.
- Questions types prises en charge : état de santé d'un équipement, liste des alertes critiques de la semaine, équipements dont le TTF est inférieur à 72 h, synthèse d'un audit récent.

Apport de l'intelligence artificielle : usage d'un LLM local en mode « agent contraint » : le périmètre des actions possibles est limité à des fonctions de lecture pré-définies, ce qui élimine le risque d'hallucination sur des données critiques et évite toute action destructrice initiée par le chatbot.

Données en entrée	Données / actions en sortie
Question en langage naturel, contexte de session, droits de l'utilisateur connecté	Réponse en langage naturel, éventuellement accompagnée d'un tableau ou d'un graphique

Critères d'acceptation

- ✓ Réponse correcte sur au moins 8 des 10 questions types du jeu de test de démonstration.
- ✓ Aucune action de modification déclenchable depuis le chatbot (lecture seule stricte).
- ✓ Temps de réponse inférieur à 5 secondes sur le lab de démonstration.

Technologies retenues : Ollama (LLM local), function calling structuré, FastAPI, composant de messagerie instantanée React.

6. Modèle de données (vue logique)

La base de données centrale (SQLite en environnement local / démonstration pour une portabilité immédiate sans dépendance de serveur SGBD tiers, compatible PostgreSQL sans modification applicative via SQLAlchemy) structure l'ensemble des informations collectées et produites par les modules.

Table	Champs principaux	Description
equipements	id, nom, ip, mac, type, os, firmware, uptime, date_decouverte	Inventaire du parc (module 4)
metriques	id, equipement_id, type_metrique, valeur, horodatage	Séries temporelles de supervision (module 2)
predictions	id, equipement_id, metrique, ttf_estime, date_calcul	Résultats du modèle de régression (module 2)
evenements_securite	id, source_ip, utilisateur, type_evenement, score_anomalie, severite, horodatage	Journal d'anomalies détectées (module 1)
audits_reseau	id, equipement_id, constat, regle_cis, criticite, correctif_propose, statut, date_audit	Résultats d'audit NetDevOps (module 3)
sauvegardes_config	id, equipement_id, contenu, hash_integrite, date_sauvegarde	Historique des configurations sauvegardées
alertes	id, type, module_origine, severite, message, statut, date_creation	Alertes consolidées tous modules confondus

Table	Champs principaux	Description
utilisateurs	id, identifiant, hash_mot_de_passe, role, mfa_active, date_creation	Comptes et droits (Administrateur/Technicien/Visiteur)
journal_audit	id, utilisateur_id, action, cible, horodatage, adresse_ip	Traçabilité des opérations sensibles

Relations principales : un équipement possède plusieurs métriques, prédictions, audits et sauvegardes de configuration (relations 1-N) ; chaque alerte référence son module d'origine et, le cas échéant, l'équipement concerné ; chaque entrée du journal d'audit référence l'utilisateur ayant réalisé l'action.

7. Exigences non fonctionnelles

Catégorie	Exigence	Niveau cible
Performance	Temps de réponse du dashboard	< 3 s pour l'affichage des indicateurs principaux
Performance	Traitement d'un audit réseau	< 30 s par équipement
Disponibilité	Disponibilité de la plateforme pendant la démonstration	≥ 99 % sur la durée de la soutenance
Sécurité	Authentification des utilisateurs	Mot de passe fort + MFA (TOTP) obligatoire pour le rôle Administrateur
Sécurité	Chiffrement des données sensibles	Secrets/identifiants chiffrés au repos, connexions internes en SSH/TLS
Sécurité	Traçabilité	Journal d'audit non modifiable par les rôles non-Administrateur
Ergonomie	Prise en main de l'interface	Navigation possible sans formation pour un technicien réseau
Portabilité	Installation de la plateforme	Déploiement complet via docker-compose up en moins de 15 minutes
Scalabilité	Nombre d'équipements supervisés	Architecture validée jusqu'à 50 équipements simulés sans dégradation notable
Conformité	Respect du RGPD sur les journaux	Anonymisation partielle des IP internes, purge automatique après durée définie
Maintenabilité	Qualité du code	Couverture de tests unitaires ≥ 60 % sur les modules IA, code documenté (docstrings)

8. Stack technique & charte graphique

8.1 Composants technologiques

Composant	Choix technologique	Justification
Langages principaux	Python 3.11+ & TypeScript / JavaScript	Python pour l'écosystème IA/réseau/FastAPI. TypeScript/React pour la solidité et la modularité du frontend.
Backend / API	FastAPI	Performances asynchrones, typage strict Pydantic, auto-génération de la documentation interactive Swagger/OpenAPI (/docs).
Frontend / Dashboard	React 18/19 (Vite + Tailwind CSS)	Restitution pixel-perfect des maquettes Stitch validées, ergonomie professionnelle « poste de commandement », fluidité SPA sans rechargement de page.
Machine Learning	scikit-learn, pandas, numpy	Standard de l'industrie, modèles légers adaptés à un lab
IA générative	Ollama (LLM local, ex. Llama 3 / Mistral)	Confidentialité (pas d'envoi de configs vers le cloud), coût nul

Composant	Choix technologique	Justification
Réseau / collecte	Netmiko, Nornir, pysnmp, psutil	Standards NetDevOps pour l'automatisation d'équipements Cisco
Base de données	SQLite par défaut (compatible PostgreSQL via SQLAlchemy)	Zéro friction d'installation, résilience maximale pour le jour de la soutenance (fichier local autonome), évolutivité immédiate vers PostgreSQL sans modifier le code.
Authentification	JWT + TOTP (pyotp)	Standard sécurisé pour API REST, MFA simple à intégrer
Conteneurisation	Docker / docker-compose	Reproductibilité du lab de démonstration, portabilité
Rapports PDF	ReportLab / WeasyPrint	Génération de documents structurés depuis Python
Maquette réseau	GNS3 / EVE-NG, VMs Linux/Windows	Environnement de test réaliste sans matériel dédié
Tests	pytest, coverage	Fiabilisation des modules avant démonstration
Versionning / CI	Git, GitHub/GitLab Actions	Traçabilité du développement, exécution automatique des tests

8.2 Charte graphique de l'interface (validée)

La charte ci-dessous est celle effectivement produite et validée sur les maquettes Stitch de l'application Sentinelle (Dashboard, Sécurité, Supervision, NetDevOps, Parc informatique, Assistant IA, Administration & Rapports). Elle fait foi pour toute implémentation de l'interface (configuration Tailwind CSS et composants modulaires React).

Palette de couleurs

Rôle	Aperçu	Code hexadécimal	Usage
Fond d'application (surface)		#101416	Canevas principal, mode sombre par défaut
Fond le plus bas (sidebar)		#0B0F11	Barre de navigation latérale
Conteneur / carte		#1D2022	Cartes KPI, panneaux, tableaux
Conteneur élevé		#272A2D	Modales, popovers, éléments survolés
Primaire — Teal		#78D8BA	Actions principales, branding, indicateur actif, lien
Secondaire — Bleu clair		#81D0F8	Métriques secondaires (ex. RAM), éléments informatifs
Tertiaire — Bleu		#A7C8FF	Accents graphiques, séries de graphiques additionnelles
Texte principal		#E0E3E6	Texte sur fond sombre (on-surface)
Texte atténué		#BDC9C3	Libellés secondaires, légendes
Bordures / séparateurs		#3E4945	Contours de champs, séparation de lignes de tableau
Statut critique / erreur		#FFB4AB	Alertes critiques, badges de sévérité élevée

Couleurs sémantiques de statut (usage transverse dans tous les modules) : vert pour un état sain/conforme, ambre pour un avertissement, rouge/rose (#FFB4AB) pour un état critique — avec un léger halo à 20 % d'opacité réservé aux alertes critiques afin d'attirer l'attention sans surcharger l'interface.

Typographie

Usage	Police	Exemple de taille
Titres / display	IBM Plex Sans (600)	40px (display), 32px (headline-lg), 24px (headline-md)

Usage	Police	Exemple de taille
Texte courant	Inter (400)	16px (body-lg), 14px (body-md)
Données & métriques	JetBrains Mono (500-600)	36px (metric-xl), 12px (label-sm, majuscules)

La police monospace JetBrains Mono est imposée pour toute donnée numérique, horodatage ou libellé de statut : elle garantit un alignement stable des tableaux lors des rafraîchissements temps réel et évite les effets de « saut » visuel.

Formes, espacement et mise en page

- Rayons d'angle : 8px par défaut (boutons, champs, petites cartes), 16px pour les conteneurs principaux et modales, forme « pilule » réservée aux badges de statut.
- Grille : grille fluide 12 colonnes ; les cartes du dashboard occupent 3 ou 4 colonnes pour maximiser la densité d'information.
- Unité d'espacement : base de 4px ; barre latérale fixe à 260px (réduite à 64px en mode icônes sur tablette) ; en-tête à 64px de hauteur.
- Profondeur : pas d'ombres portées marquées — la hiérarchie visuelle se fait par couches tonales successives et bordures fines à faible opacité (rgba blanc 8 %).
- Mobile : empilement vertical des colonnes, cartes en pleine largeur, marge latérale réduite de 32px à 16px.

Principes directeurs de l'identité visuelle

L'interface adopte une esthétique « Corporate / Modern », proche des outils d'observabilité professionnels (Datadog, Grafana), avec une sensation de « poste de commandement opérationnel » : autorité, précision technique, densité maîtrisée de l'information, sans surcharge visuelle. Le logo Sentinelle (bouclier stylisé formé d'un réseau de nœuds) reprend la couleur primaire teal en dégradé sur fond sombre, cohérent avec l'ensemble de la charte.

9. Gestion des utilisateurs, rôles et sécurité applicative

9.1 Matrice des droits par rôle

Fonctionnalité	Visiteur	Technicien	Administrateur
Consultation du dashboard	Oui	Oui	Oui
Consultation des rapports	Oui (limité)	Oui	Oui
Lancement d'un audit réseau	Non	Oui	Oui
Application d'un correctif proposé	Non	Oui (avec validation)	Oui
Gestion des comptes utilisateurs	Non	Non	Oui
Consultation du journal d'audit	Non	Non	Oui
Configuration des notifications	Non	Oui (ses propres alertes)	Oui (globale)

9.2 Exigences de sécurité applicative

- Mots de passe hachés (bcrypt/argon2), jamais stockés en clair.
- Authentification à double facteur (TOTP) obligatoire pour le rôle Administrateur.
- Jetons d'accès JWT à durée de vie limitée, renouvellement par jeton de rafraîchissement.
- Limitation du nombre de tentatives de connexion (protection brute-force sur la plateforme elle-même).
- Chiffrement des identifiants d'équipements réseau stockés (Fernet/AES) et non en clair en base.
- Journal d'audit horodaté, en écriture seule pour les rôles non-Administrateur.

10. Plan de tests et recette

10.1 Niveaux de test

Niveau	Objet	Exemple
Unitaire	Fonctions isolées de chaque module	Vérifier le calcul du score d'anomalie sur un jeu de données connu
Intégration	Interaction entre modules et base de données	Vérifier qu'une alerte Sécurité crée bien une entrée dans la table alertes
Système	Plateforme complète sur le lab GNS3/EVE-NG	Scénario complet : découverte, supervision, alerte, notification
Acceptation (recette)	Conformité aux critères d'acceptation du §5	Rejeu des scénarios de soutenance devant tuteur avant le jour J

10.2 Scénarios de test représentatifs

- **Sécurité :** simulation d'une attaque par force brute SSH (ex. via hydra en environnement isolé) et vérification de la détection et de l'alerte associée.
- **Supervision :** saturation progressive du disque d'une VM de test et vérification de la précision de la prédiction de Time-To-Failure.
- **NetDevOps :** introduction volontaire d'une faille de configuration (mot de passe en clair) sur un switch de laboratoire et vérification de sa détection par l'audit.
- **Gestion de parc :** ajout d'un nouvel équipement sur le réseau simulé et vérification de sa découverte automatique dans l'inventaire.
- **Sécurité applicative :** tentative de connexion avec des identifiants incorrects répétée pour vérifier le verrouillage du compte.

11. Gestion de projet

11.1 Méthodologie retenue

Le projet est conduit selon une démarche Agile adaptée (sprints de deux semaines), avec un backlog de fonctionnalités priorisées par valeur pédagogique et complexité technique. Chaque sprint se termine par une démonstration courte permettant d'ajuster les priorités du sprint suivant, à la manière d'une revue de sprint Scrum simplifiée.

11.2 Planning prévisionnel (exemple sur 5 mois)

Phase	Durée	Contenu
Phase 1 — Cadrage	2 semaines	Cahier des charges, choix technologiques, maquette d'architecture, mise en place du lab GNS3/EVE-NG
Phase 2 — Socle technique	3 semaines	Base de données SQLite/SQLAlchemy, API FastAPI, authentification/RBAC, squelette frontend React + Vite + Tailwind
Phase 3 — Module Supervision	2 semaines	Collecte SSH/SNMP, modèle de régression, prédiction TTF
Phase 4 — Module Sécurité	2 semaines	Parsing des journaux, Isolation Forest, alertes
Phase 5 — Module NetDevOps	3 semaines	Netmiko, intégration LLM local, audit CIS, sauvegarde de configuration
Phase 6 — Gestion de parc & Reporting	2 semaines	Découverte SNMP, score de santé, rapports PDF, notifications
Phase 7 — Assistant IA	1,5 semaine	Intégration du chatbot en lecture seule sur les données consolidées
Phase 8 — Tests & recette	2 semaines	Scénarios de test, corrections, rejeu de la démonstration

Phase	Durée	Contenu
Phase 9 — Préparation soutenance	1,5 semaine	Mémoire technique, support de présentation, répétitions

11.3 Budget prévisionnel (projet de laboratoire, sans matériel dédié)

Poste	Nature	Estimation
Licences logicielles	Python, SQLite/PostgreSQL, React, Vite, Tailwind CSS, Ollama, Docker : composants 100 % open-source et gratuits	0 €
Environnement de simulation	GNS3/EVE-NG (gratuits), images Cisco (licence pédagogique/labo)	0 € à faible coût selon licence labo
Infrastructure de calcul	Poste de développement existant ou VM d'établissement	Déjà disponible (établissement)
Nom de domaine / hébergement démo (optionnel)	Si démonstration en ligne	~10-15 €/an (optionnel)
Temps projet	Charge de travail étudiant	~5 mois, mi-temps projet estimé

Le projet est volontairement construit avec une stack 100 % open-source afin de rester réalisable sans budget dédié, condition réaliste d'un projet de BTS.

11.4 Analyse des risques

Risque	Probabilité	Impact	Mesure de mitigation
Instabilité du LLM local (temps de réponse, exactitude)	Moyenne	Moyen	Prompts structurés et testés en amont, réponse en JSON validée, repli sur règles statiques si échec
Complexité sous-estimée de l'intégration Netmiko/GNS3	Moyenne	Élevé	Prototype dès la phase 1, marge de 20 % sur le planning des modules réseau
Faux positifs/négatifs du modèle d'anomalies	Élevée	Moyen	Calibrage sur jeux de données simulés variés, seuils ajustables par l'utilisateur
Indisponibilité du matériel réseau physique	Faible	Élevé	Recours systématique à GNS3/EVE-NG comme solution de secours
Retard sur la charge de développement (solo/binôme)	Moyenne	Élevé	Priorisation stricte du backlog, fonctionnalités « bonus » en fin de liste
Faille de sécurité sur la plateforme elle-même	Faible	Élevé	Revue de sécurité dédiée avant soutenance, tests d'intrusion basiques internes

12. Livrables attendus

- Code source complet, organisé par module, versionné sur dépôt Git avec historique de commits.
- Base de données de démonstration pré-remplie (jeu de données simulé) et scripts de migration.
- Documentation technique (architecture, modèle de données, guide d'installation Docker).
- Cahier de tests et rapport de recette (résultats des scénarios du §10).
- Mémoire technique de soutenance intégrant ce cahier des charges maturé.
- Support de présentation orale (20 minutes) et script de démonstration live.
- Environnement de laboratoire GNS3/EVE-NG exporté (topologie réutilisable).

13. Perspectives d'évolution (hors périmètre de soutenance)

- **Orchestration avancée :** intégration d'Ansible pour l'application automatisée et versionnée des correctifs validés.
- **Mode SaaS multi-tenant :** isolation des données par client/organisation pour une exploitation commerciale.
- **Supervision Cloud :** extension de la collecte aux environnements AWS/Azure/GCP via leurs API natives.
- **Observabilité de la plateforme :** intégration Prometheus/Grafana pour le monitoring de la solution elle-même.
- **SOAR léger :** automatisation de réponses à incident (blocage temporaire d'IP, isolation réseau) avec validation humaine.
- **Application mobile :** client léger consommant l'API REST pour les alertes en mobilité.
- **Amélioration du modèle IA :** passage à des modèles de séries temporelles plus avancés (Prophet, LSTM) si le volume de données le justifie.

14. Glossaire

Terme	Définition
AIOps	Artificial Intelligence for IT Operations — usage de l'IA pour automatiser et améliorer l'exploitation informatique
Isolation Forest	Algorithme de détection d'anomalies non supervisé basé sur le partitionnement aléatoire des données
Time-To-Failure (TTF)	Délai estimé avant qu'une ressource n'atteigne un seuil critique de saturation
CIS Benchmark	Référentiel de bonnes pratiques de sécurisation des systèmes et équipements réseau
Netmiko / Nornir	Bibliothèques Python d'automatisation de la connexion et de la configuration d'équipements réseau
LLM	Large Language Model — modèle de langage capable de comprendre et générer du texte
Ollama	Outil permettant d'exécuter des modèles de langage en local, sans dépendance à un service cloud
RBAC	Role-Based Access Control — gestion des droits d'accès fondée sur des rôles utilisateurs
JWT	JSON Web Token — jeton d'authentification utilisé pour sécuriser les échanges avec l'API
MFA	Multi-Factor Authentication — authentification à plusieurs facteurs (ex. mot de passe + code TOTP)

Document réalisé dans le cadre de la préparation à l'épreuve E6 — Projet AIOps, BTS SIO (SISR) / CIEL / CPR.